

Implementation Plan for Multiturn Guided Funnel Framework (Mid-size Car Sales)

1. Core Modules Overview

1.1 Dialogue State Management Module (DST)

- **Purpose**: Maintain up-to-date conversation context, sales funnel stage, collected slot values, and recommendations history.
- **Key Fields**:
 - `chat_history`: Turn-by-turn conversation messages.
 - `filled_slots`: Dict of slot name → value (e.g., `budget`, `seat_count`, `usage`).
 - `recommendations`: List of product IDs/names suggested.
- **Technologies**: `Python TypedDict`, `LangGraph Stateful Graph`, `LangChain Memory`.
- **Potential Issues**: State loss, incomplete slot schema, misalignment between slot keys and schema.

1.2 Dialogue Manager (DM)

- **Purpose**: Decide the next system action based on the current state.
- **Key Functions**:
 - Route conversation flow (`Think` step).
 - Enforce business funnel logic.
- **Technologies**: `LangGraph Conditional Edges`, `LangChain PromptTemplate`, LLM with structured output.
- **Potential Issues**: Misclassification of user intent, deviation from funnel logic.

1.3 Slot Filling Engine

- **Purpose**: Gather essential purchase requirements (e.g., budget, seating capacity, fuel type, brand preference).
- **Key Features**: Required/optional flags, multi-value slots, clarifying question templates.

- **Technologies**: LLM-based entity extraction, prompt chaining, LangChain tool calling.
- **Potential Issues**: Ambiguous user input, synonyms mapping, missing clarifying steps.

1.4 Recommendation Engine

- **Purpose**: Filter vehicle catalog and generate persuasive recommendations.
- **Approach**:
 1. Basic version: Programmatic filtering.
 2. Advanced version: RAG-based semantic retrieval + LLM synthesis.
- **Technologies**: `Pandas`, `FAISS`/`Chroma`, `LangChain RetrievalQA`.
- **Potential Issues**: Poor relevance ranking, hallucinated specs.

1.5 Prompt Chaining ("Think, Then Act")

- **Purpose**: Split decision-making (Think) and user-facing generation (Act) into separate steps.
- **Technologies**: `LangChain ChatPromptTemplate`, structured output with Pydantic models.
- **Potential Issues**: Template variables mismatch, too broad prompts causing drift.

1.6 Vehicle Knowledge Base

- **Purpose**: Store detailed specs of mid-size cars for search and retrieval.
- **Data Structure**: CSV with columns for brand, model, seat_count, fuel_type, price_range, features, etc.
- **Technologies**: `Pandas` for CSV loading, vector DB for RAG.
- **Potential Issues**: Data maintenance, inconsistent schema across brands.

1.7 Long-Term Memory & Personalization

- **Purpose**: Persist user data across sessions for personalized experience.

- **Approach**: Store DialogueState in Redis or PostgreSQL keyed by `user\_id`.
- **Technologies**: `SQLAlchemy`, `Redis`, LangGraph checkpointing.
- **Potential Issues**: Privacy compliance, reliable user identification.

1.8 Error Handling & Conversational Repair

- **Purpose**: Handle off-topic queries, misunderstandings, or user frustration.
- **Strategies**:
 - Rephrase questions.
 - Offer multiple-choice options.
 - Allow graceful interruption handling.
 - Escalate to human agents if needed.
- **Technologies**: Sentiment analysis, fallback routing in LangGraph.
- **Potential Issues**: Overuse of repair prompts leading to user annoyance.

2. Technology Stack

- **Core**: Python, LangChain, LangGraph
- **LLM Providers**: OpenAI / Anthropic
- **Data Processing**: Pandas
- **Vector Store**: FAISS / Chroma
- **Persistence**: Redis / PostgreSQL
- **Structured Output Validation**: Pydantic

3. Potential Challenges

1. **LLM Routing Errors** — May lead to skipped funnel steps.
2. **Data Freshness** — Vehicle knowledge base must be regularly updated.

3. **Prompt Robustness** — Balancing persuasive sales tone with natural dialogue.
4. **Synonym & Ambiguity Handling** — Ensuring slot-filling understands varied phrasing.
5. **Performance** — RAG and LLM calls may slow responses; caching needed.

4. Implementation Phases

1. **Data Preparation**
 - Define slot schema for mid-size car sales.
 - Build initial vehicle knowledge base CSV.
2. **Core Framework Setup**
 - Implement `DialogueState` structure.
 - Initialize LangGraph stateful workflow.
3. **Routing & Prompt Chaining**
 - Build `route\_action` function (programmatic first, then LLM-based).
 - Implement `Think` and `Act` prompt templates.
4. **Slot Filling**
 - Implement slot extraction logic.
 - Build clarifying question generation.
5. **Recommendation Logic**
 - Implement basic filtering.
 - Integrate RAG pipeline.
6. **Error Handling**
 - Add off-topic, repair, and escalation flows.
7. **Persistence Layer**

- Add state saving and loading for long-term memory.

8. ****Testing & Optimization****

- Simulate multiple user journeys.
- Tune prompts, slot schema, and retrieval quality.
